Leader election

Abbiamo visto come nei sistemi distribuiti la perfetta simmetria sia una delle configurazioni più problematiche che possano capitare.
Per romperla ci si è posti il problema della Leader Election, che generalmente richiede di portare la rete in una configurazione finale in cui esista un'unico leader che poi vada a gestire le varie operazioni

Si può pensare la Leader Election come un modo per forzare la restrizione di Unique Initiator

Assunzioni e restrizioni

- $R \rightsquigarrow TR, BL, CN$
- Unique Id $\rightsquigarrow \forall x,y \in V \;\; id(x) \neq id(y)$

Oss

Sotto la sola assunzione R non è possibile risolvere la task senza ricorrere ad una soluzione probabilistica

Configurazione iniziale e finale

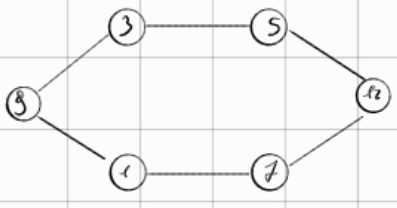
- $P_{init} = \forall x \in V : state(x) = Init$
- $P_{final} = \exists! x \in V : state(x) = Leader \wedge \forall y \neq x : state(y) = Follower$

Andiamo ora a studiare la leader election nelle varie topologie

Topologia ad Anello

Ci troviamo all'interno di un rete $R = (V, C)$ dove

- $V = \{x_1, \dots, x_n\}$
- $C = \{(x_i, x_{i+1}) : \forall i = 1, \dots, n-1\} \cup \{(x_n, x_1)\}$



Inoltre ogni nodo è in grado di distinguere tra il vicino destro ed il vicino sinistro

Bisogna notare che gli id non siano globalmente consistenti, ovvero che se $id(x_i) = k$ non è detto che $id(x_{i+1}) = k+1$

Vediamo ora 3 protocolli per risolvere la task

Protocollo All The Way

L'idea dietro al protocollo consiste nell'inviare ogni etichetta id(x) ad ogni nodo

Idea

- Quando un nodo $x \in V$ si attiva manda un messaggio id(x) in una direzione concorde a tutti
- Alla ricezione del msg il nodo
  - invia la propria etichetta se non l'ha inviata prima,
  - aggiorna il proprio minimo
  - inoltra il msg ricevuto

Grazie al vincolo di TR esisterà un istante in cui ogni nodo vedrà gli id di ogni altro nodo e quindi potrà determinare se sia o meno il Leader

Il problema ora è capire quando un nodo possa entrare nello stato quiescente in quanto a causa dei ritardi dei messaggi non basta che un nodo x riceva il msg id(x). $n = |V|$

Una buona idea è quella di contare i msg in arrivo e, nel momento in cui si riceve l'n-esimo, allora il nodo si blocca.

Ma n non è conosciuto a priori dai nodi, quindi per risolvere il problema si aggiunge un contatore all'interno del msg che si incrementa ad ogni inoltro.

In questo modo quando il nodo x riceverà id(x) con n riuscirà a capire quando avrà visto tutte le etichette di ogni nodo ↳ Fa il giro dell'anello

Conclusioni

Grazie a TR ogni nodo prima o poi riceverà tutte le informazioni di cui ha bisogno per calcolare l'etichetta minima

Complessità

Message Complexity:

Prendiamo l'arco e = (x,y)

Visto che ogni nodo genera un solo msg che attraversi tutta la rete, sull'arco e passano n msg

Estendendo questo ragionamento per ognuno degli n archi della rete abbiamo

$$M(\text{All-the-Way}) = \mathcal{O}(n^2)$$

Se si vuole calcolare invece la bit complexity bisogna vedere che ogni msg pesa $\log(id(x)+n)$ bit

Quindi abbiamo che

$$M_{bit}(\text{All the Way}) = \mathcal{O}(n^2 \log(id(x)+n))$$

Time Complexity:

Sotto le assunzioni di Sincronia e Bound dei ritardi si ha un tempo di terminazione

$$T(\text{All The Way}) = \mathcal{O}(n)$$

Anche considerando il caso peggiore dove i nodi vengono svegliati sequenzialmente e solo quando ricevono il primo msg

Da quel momento in poi bastano n istanti di tempo per far fare il giro della sua etichetta in n istanti

Protocollo Asincrono

Possiamo notare che in All the Way vengono inviati più msg del necessario

Infatti se un nodo x con id(x) riceve il msg id(y) e id(y) > id(x),

Allora x non avrà motivo di inoltrare id(y) in quanto so che questo non sarà mai l'etichetta minima

Idea

In merito a quest'osservazione si modifica All the Way in modo tale che

- Ogni nodo x blocca tutti gli id con valore maggiore del valore che x ritiene il minimo

Terminazione

Con questa modifica notiamo che nel momento in cui un nodo riceva la propria etichetta, questo lo rende sicuramente il Leader

Non è certo che attraversino tutta la rete

Per quanto riguarda gli altri nodi non si può più fare affidamento ad un contatore nei msg
Questo si risolve eseguendo una fase di broadcast in cui il Leader notifica tutti gli altri nodi

Complessità

Message Complexity:

Grazie a questa modifica studiare la msg complexity è più complesso.
Per questo studiamo il worst case dove i nodi sono ordinati con etichette crescenti,

In questo modo il msg_{id_1} sarà il primo ad essere inviato e farà il giro dell'anello, → n msg
L'id_2 farà lo stesso giro ma verrà fermato solo dal nodo con id_2 e così via → n-1 msg

In questa situazione quindi abbiamo

$$M(\text{LeaderFinder}) = n + \sum_{i=1}^{n} i = n + \frac{n(n+1)}{2} = \mathcal{O}(n^2)$$

Il caso migliore invece si ha quando i nodi sono ordinati secondo un'ordine decrescente,
Infatti tutti i msg vengono inoltrati una sola volta mentre l'unico che viene inoltrato n volte sarà quello del

Futuro Leader

↳ # di msg: $n + (n-1) + n$ → Contando anche il Broadcast

Protocollo Stage Technique

Abbiamo visto come anche in All The Way i msg possano venir inoltrati più del necessario
Un'ulteriore miglioramento è quello di limitare la distanza a cui trasmettere i msg ed avere poi
dei feedback per capire se bloccare o no i msg

Idea

Mancanza di Bound dei ritardi e Sincronia

La logica del protocollo si basa sulla divisione dell'esecuzione in stage, i quali possono anche avvenire in contemporanea
Per semplicità gli stage verranno mostrati come sincroni per poi fare le dovute precisazioni

Ogni nodo attivo $x \in V$ allo stage i esegue le seguenti operazioni:

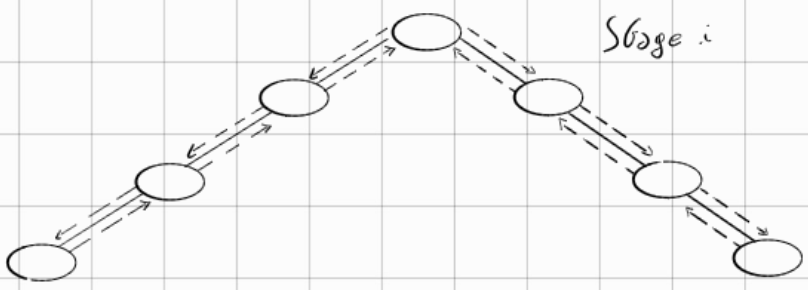
a) Invia a dx e sx $id(x)$ fino a distanza 2^{i-1}
b) Il msg viene inoltrato finché non incontra un'etichetta minore

c) Se il msg arriva a distanza 2^{i-1} allora forma indietro sotto forma di feedback

d) Se il nodo x riceve il msg di feedback da entrambi i lati allora può passare allo stage i+1

Detto questo elenchiamo ora le regole che descrivono il cambio di stato dei nodi ed il loro comportamento

1) Se un nodo candidato riceve un id minore del suo allora entra nello stato defeated
2) Se un nodo non riceve entrambi i feedback entra nello stato defeated
3) Un nodo defeated può solo inoltrare msg
4) Un nodo candidato diventerà Leader solo se riceverà il suo msg dal lato opposto a quello a cui lo ha inviato
5) Il Leader invierà un msg di broadcast che fa diventare ogni nodo follower





Oss

Non essendoci sincronia può capitare che i nodi si trovino in stage diversi

Può arrenire in fatti che un nodo venga sconfitto da un msg proveniente da uno stage k mentre attende il suo feedback dello stage k-1.

In questo caso il nodo verrà semplicemente bloccato come descritto nel protocollo

Correttezza e Terminazione

È facile vedere come in un tempo finito verrà svegliato il nodo con etichetta minima,

A quel punto dopo aver eseguito i stage t.c. $2^i \geq n \Rightarrow i = O(\log n)$ il nodo riceverà nuovamente il suo msg e quindi verrà eletto → Correttezza

Dato che ogni stage dura tempo finito allora il protocollo termina in tempo finito ⇝ Terminazione

Complessità

Per calcolare la Message complexity contiamo ora il numero di msg inviati in ogni stage

Definiamo la quantità:

$n_i := |\{\text{nodi che superano lo stage } i-1\}|$

Possiamo notare come x supera lo stage $i-1$ solo se il suo id(x) soddisfa 2^{i-2} nodi a dx e sx

Per cui

$$n_i \leq \frac{n}{2^{i-2}+1}$$

Sia $i > 1$.

Nel caso peggiore abbiamo

- Msg forti:

  Questi msg inviati nello stage i percorrono 2^{i-1} salti in entrambe le direzioni

  Essendoci al più n_i candidati questi sono $n_i \cdot 2 \cdot 2^{i-1}$

- Msg Feedback:

  Per ogni nodo che supera lo stage i vengono trasmessi $2 \cdot 2^{i-1}$ Feedback

  Per ogni nodo che invece non lo supera questo riceve al più un msg di Feedback

  Quindi abbiamo un totale di $n_{i+1} \cdot 2 \cdot 2^{i-1} + (n_i - n_{i+1}) 2^{i-1}$ msg

  ↳ superano i ↳ soddisfatti

Per cui abbiamo che $\forall i > 1$ il numero di msg è al più:

$$n_i \cdot 2 \cdot 2^{i-1} + n_{i+1} \cdot 2 \cdot 2^{i-1} + (n_i - n_{i+1}) 2^{i-1} = 2^{i-1}(3n_i + n_{i+1}) \leq$$

$$\leq 2^{i-1}\left(3\left\lfloor \frac{n}{2^{i-2}+1} \right\rfloor + \left\lfloor \frac{n}{2^{i-1}+1} \right\rfloor\right) <$$

$$< \frac{3n\,2^{i-1}}{2^{i-2}+1} + \frac{n\,2^{i-1}}{2^{i-1}+1} <$$

$$< \frac{3n\,2^{i-1}}{2^{i-2}} + \frac{n\,2^{i-1}}{2^{i-1}} = 6n + n = 7n$$

Per il 1° sfase invece abbiamo nel caso peggiore che tutti i nodi siano inizialtori

In questo caso si ha che

$$n_2 \leq \frac{n}{2^0+1} = \frac{n}{2}$$

Allora i nodi che superano il primo sfase inviano 2 msg "forth" e 2 msg "feedback"

Per un totale di $n_2 \cdot 4$ msg

Per quelli che non lo superano invece si hanno almeno 2 msg "forth" e 1 msg "feedback"

Per un totale di $3(n-n_2)$ msg

Quindi i msg inviati nello sfase $i=1$ sono al più

$$n_2 4 + 3(n-n_2) = n_2 + 3n < 4n$$

Combinando i 2 conti abbiamo che

$$M(\text{Sfase}) \leq 4n + \sum_{i=2}^{\log_2 n} 7n = 4n + 7n \log n = O(n \log n)$$